

SIH 2026

Four Solutions, Simply

Four problem statements. Four solutions in plain language.
Basic roadmap for each.

SIH26156 ULPF / NTRO · SIH26076 Mausam / IMD · SIH26087 NCCT · SIH26138 Green Fleet

October 2026

SIH 2026 — Four Solutions, Simply

Four problem statements. Four solutions in plain language. Basic roadmap for each.

Deadline: **5 October 2026**.

#	PS	Organisation	Problem in one line	Open slots
1	SIH26156	NTRO	Every device sends logs in a different format, so security teams waste weeks writing parsers and lose data silently	323
2	SIH26076	IMD (Mausam app)	The weather app shows data, but never tells 8 different kinds of users what to actually <i>do</i>	178
3	SIH26087	NCCT (Cooperation Ministry)	2.27 lakh people trained in a year, but no way to prove what they learned or match them to jobs	402
4	SIH26138	Egreen Quanta	Shipping companies must cut fuel, cost and emissions at once, under IMO rules — too many variables to solve by hand	375

1. SIH26156 — LOGSMITH (NTRO, log normalisation)

The problem. A firewall, a server and a CCTV camera all write logs in completely different formats. Any team wanting to analyse them must write a separate parser for each one — days of work per device. Worse, parsers quietly drop or rename fields, and nobody notices until an incident review goes wrong.

Our solution — LOGSMITH. A pipeline that takes any log, converts it to one standard format (OCSF, the open cybersecurity standard), and keeps a sealed copy of the original log inside every output record.

The key idea: **we can prove nothing was lost.** Press one button and the system rebuilds the original log from our output and compares checksums. If they match, the conversion was lossless. Anyone claiming "lossless" can say it — we can *show* it.

Two extras:

- **Quick setup** — drop 300 sample lines from a new device, and the system figures out the format itself and proposes the mapping, with a confidence score. You approve it with one click.
- **Change detector** — if the company updates its device software and renames a field, LOGSMITH notices, stops trusting the old rule, and flags it. Without this, security alerts go silently blind.

Basic roadmap (7 weeks)

Week	Work	Done when
1	Read formats, lock OCSF standard version, collect sample logs	5 formats parsing correctly
2	Core pipeline: read log → convert → standard format → store	One end-to-end path works

Week	Work	Done when
3	Mapping registry with version control + auto-setup engine	New device onboarded in under 10 min
4	Change detector + dashboard	Renamed field is caught automatically
5	Web screens (3 only): setup, dashboard, event inspector	Full journey clickable
6	Packaging, offline mode test, performance measurement	Runs with zero internet, speed measured
7	Demo script, video, slides, judge prep	2-min video + 5 slides ready

Demo moment: paste an unfamiliar log line → system proposes a mapping → press VERIFY → watch the original log rebuild from the output, checksums match → rename one field → watch the change detector catch it live.

2. SIH26076 — Mausam (IMD personalised homepage)

The problem. The official statement lists eight users — a patient, a runner, a surfer, a traveller, a parent, a farmer, a commuter, an event planner. Each needs different information. The official text is *only* that list, with no background or dataset — so the real difficulty is figuring out what to build from almost nothing.

Our solution — Mausam. Not a new weather app. The personalised home screen, sitting on IMD's official data (28 documented APIs), giving each user a short list of *actions*, not charts.

- "Heat wave warning + 44°C at your station → avoid outdoor work 12–4 pm."
- "Fog warning on your commute route → leave 20 minutes early."
- "Rain likely in 4 of 7 days → carry a raincoat in London."

The differentiator: **every number can explain itself.** Tap any number and it shows the exact IMD data source, which station, what time it was issued, and how old it is. No weather app in India does this — and for a government app, trust is the actual problem.

Also built in from the start, not as an afterthought:

- **Works offline** — full personal forecast cached on the phone, loads on slow 2G.
- **Regional languages** — warning text reviewed by humans, not machine-translated.
- **Accessibility** — screen-reader friendly, keyboard navigable, high-contrast severe-weather states.
- **Honest gaps** — the statement asks for pollen and AQI. IMD doesn't publish pollen. Instead of inventing a number, the app says "data not available" and explains why. That honesty is the point.

Basic roadmap (7 weeks)

Week	Work	Done when
1	Apply for IMD API access, map all 28 endpoints, list what each persona needs	Access works or fallback confirmed

Week	Work	Done when
2	Data layer: fetch, cache, track age of every value	All personas' data flowing with timestamps
3	Rules engine that turns data into actions	Every action traceable to its source data
4	Core app screen + the "explain this number" feature	Working, offline-capable
5	Add personas one by one (8 total) + languages + accessibility	All 8 personas working
6	Compare against 124 years of IMD rainfall history	"Normal vs today" comparison visible
7	Demo script, slides, judge prep	Full demo rehearsed

Demo moment: throttle the browser to slow 2G → app still loads instantly from cache → a fog warning appears with a real action → tap the temperature → the screen shows the IMD station, issue time and age of that number.

3. SIH26087 — SAHAKAR SETU (NCCT, Ministry of Cooperation)

The problem. NCCT runs 20 institutes and trained 2.27 lakh people in one year. Its systems are manual and disconnected. Trainees finish courses but cannot prove what they can do, and employers cannot trust certificates they have never checked. So training happens, but outcomes are invisible.

The statement lists eleven features. Building eleven shallow tabs is the trap.

Our solution — SAHAKAR SETU. Focus on the one gap that matters: **between finishing a course and getting a job.**

- **Skills backed by proof, not claims.** A trainee is "good at dairy record-keeping" only because of specific evidence — which questions they answered correctly, what practical work they submitted. No evidence, no claim. Resumes are input; evidence is truth.
- **Matching that explains itself.** A Dairy Supervisor job needs 6 skills. The system shows exactly which ones the trainee has proven, which they lack, and **which NCCT course at which institute closes each gap** — in a realistic order.
- **It learns from hiring.** When employers say hired / not hired and why, the system reweights which skills actually matter. A static job portal structurally cannot do this. It is the strongest demo moment in the whole project.
- **Certificates anyone can verify.** Each certificate gets a digital fingerprint and a public verification page — the employer checks it in one click, no phone call.
- **Offline and rural-first.** Modules, quizzes and attendance work without internet and sync later. Content in Indian languages. Institute computers are shared devices, so no trainee needs their own phone.
- **Attendance honestly.** QR scan with staff confirmation is the default. Face recognition is optional, only with consent, processed on the device, never stored, deletable. We lead with the privacy answer, not the technology — a judge will ask, and this is the honest answer.

Name inspired by NCCT's own motto: "**Sahakar se Samridhhi**" (prosperity through cooperation).

Basic roadmap (8 weeks — largest of the four)

Week	Work	Done when
1	Get 3 real course syllabi, map skills, define the skill graph	Graph covers 3 flagship courses
2	Trainee + institute profiles, registration, login	Trainee can log in and see profile
3	Course player + quizzes, offline download	Module works offline
4	Scoring engine: evidence → skill level with confidence	Every quiz produces a scored skill
5	Employer side: post a job, see matched trainees with reasons	Job → explainable matches
6	Gap planner: missing skill → exact course + institute	Full gap-closure path shown
7	Certificates + public verification page; QR attendance	Certificate verifiable by anyone
8	Outcome capture + reweighting, demo script, slides	Full loop demoable

Demo moment: open a trainee's profile → see the gap to a target job → tap a gap → the exact 3-week course appears → complete a 5-question quiz live → watch the skill score fill in with a confidence range and a certificate fingerprint → employer sees the explained match → employer rejects with a reason → watch the admin dashboard reweight itself live.

4. SIH26138 — SEAWARD (Egreen Quanta, shipping fuel + emissions)

The problem. This is a **shipping** problem, not a road-vehicle problem. A shipping company must decide: which ships, how big, what speed, which fuel (LNG, methanol, hydrogen, ammonia), whether to plug into shore power at berth — while cutting fuel cost, emissions and carbon intensity, without missing cargo deadlines. Far too many combinations to solve by hand.

There is a real regulation driving all of this: the **IMO Carbon Intensity Indicator (CII)**, which grades each ship A to E. Fuel-efficient but badly rated ships are still a compliance failure.

Our solution — SEAWARD. Four parts:

1. **Physics first, AI second.** Fuel use follows a real physical relationship with speed and conditions. We build that physics ourselves — it is small, explainable, and correctly predicts ships and speeds it has never seen. Then AI is used only to correct what physics misses (weather, hull fouling, engine wear). This is the opposite of throwing a black-box model at a CSV. It is also why our numbers hold up outside the training range, which is exactly what fleet planning needs.
2. **Optimise for the regulation, not just fuel.** We implement the actual IMO formula and grade every scenario A–E. The goal is not "least fuel" — it is "best fleet that stays compliant and on schedule."
3. **Honest "quantum-inspired".** No quantum computer is used or needed. We implement classical optimisation algorithms inspired by quantum ideas, and — critically — we solve small fleet problems **exactly** with standard integer programming and compare. If our method cannot match the exact answer on 6 ships, we do not claim quantum advantage. Judges trust teams that benchmark against a baseline.

4. **The regulatory cliff.** A timeline slider 2024 → 2035. As rules tighten, today's optimal fleet visibly slides into D/E rating territory. Then the system re-optimises and shows the cheapest compliant fleet — fuel switch, slower speeds, shore power — with the cost. *"Your optimal 2026 fleet is illegal by 2030. Here is the compliant one, and here is what it costs."*

Data honesty. Real ship-level fuel data is anonymised by IMO and not public. So we build a documented physics simulator as our testbed — equations and assumptions published — and treat any real data request as a bonus. Saying this plainly is far stronger than claiming data we don't have.

Basic roadmap (8 weeks)

Week	Work	Done when
1	Ship physics: power, drag, resistance. Write it down	Equations documented
2	Build the simulator (fleet, voyages, weather) + CII formula	Runs a full voyage, grades A-E
3	Fit per-ship-type parameters; calibrate	Predictions match simulator
4	AI residual layer; show it beats physics-only	Measured improvement table
5	Optimiser + exact integer-programming baseline; small-fleet test	Both solve the same test fleet
6	Alternative fuels, shore power, schedule constraints in scenarios	Scenario comparison works
7	Frontend: map, comparison charts, regulatory-cliff slider	Demo path clickable
8	Benchmarks, demo script, slides	Full demo rehearsed

Demo moment: load a fleet → run optimisation → comparison charts appear → drag the timeline slider to 2030 → watch the fleet's ratings collapse from A/B into D/E → press re-optimize → watch a compliant mix appear with fuel switch, slower speeds and shore power, alongside cost per tonne of CO₂ saved.

5. One team, four projects — overall plan

Each solution follows the same seven beats, so a team that learns one has learned all four:

Understand → Build the honest core → Add the intelligence → Build only the demo screens → Connect and package → Test → Rehearse.

Priority	Pick this if your team is strong at	Room left
PS1 LOGSMITH	Parsers, back-end, security	323 slots
PS2 Mausam	Front-end, APIs, mobile, design	178 slots — most crowded
PS3 SAHAKAR SETU	Full-stack, databases, Hindi content	402 slots — least crowded
PS4 SEAWARD	Maths, Python, optimisation, simulation	375 slots

Team of six: Frontend · Back-end ×2 · Data/domain · AI/optimisation · DevOps + one person for research, slides and demo script.

Build for real vs mock: anything the demo touches must genuinely work. Anything beyond it — multi-server scale, full syllabus coverage, production login — gets labelled "future scope" on the slide. Judges punish fake depth far more than honest scope.

Three rules that decide these projects

1. **Build the demo moment first.** Decide what the audience will gasp at, then build only what it needs. Every other screen is optional.
2. **Measure, don't claim.** Every number in the slides should come from your own test run. If you cannot measure it, write "to be measured in pilot" instead of a percentage.
3. **Say "we don't know" once, with a plan to find out.** In a hackathon that sentence buys more trust than any feature.